

Extended Abstract

Motivation Web agents suffer from low task completion rates due to the complexity of multi-action reasoning required for web navigation tasks. Current state-of-the-art approaches achieve only 30.3% success on WebArena (DigiRL + Llama-8B), while baseline LLMs achieve 13.1% success rate (GPT-4o). The core challenge lies in the conflicting objectives that agents face: they must simultaneously think elaborately and plan for long-horizon objectives while only outputting immediate next actions to transition pages. Moreover, agents are frequently overwhelmed by information and fail to take the right actions even with high-level plans already made.

Method We propose Harlequin, a hierarchical reinforcement learning framework that leverages the hierarchical structure of natural language instructions and web-based interaction tasks by modeling agent behavior as a two-level Markov Decision Process. Our approach decomposes web navigation into strategic planning and precise execution through two specialized policies: a high-level policy that maps task instructions and interaction history to subgoals expressed as mid-level textual directives, and a low-level policy that conditions on the current environment state and active subgoal to generate primitive actions executable in the web interface. We enhance this framework with Group Relative Policy Optimization (GRPO) for efficient online learning, eliminating the need for separate value models and reducing GPU requirements by approximately 50% compared to PPO-based approaches.

Implementation Our implementation consists of three key components: (1) A novel goal decomposition pipeline that uses inverse trajectory segmentation to automatically annotate existing flat trajectories with hierarchical subgoals, employing Monte Carlo optimization to achieve optimal segmentation quality; (2) Harlequin-Base (HBC), a hierarchical behavioral cloning system that provides supervised initialization on decomposed demonstrations; and (3) Full Harlequin with GRPO applied to both hierarchical policies for adaptive online learning. We enhanced the NNetNav dataset from 3,201 trajectories to over 100,000 hierarchically annotated examples with 4.75/5.0 subgoal success rating.

Results Harlequin achieves 41.3% average success rate on WebArena, representing a significant improvement over flat RL approaches and establishing a new state-of-the-art for open-source web agents. Our hierarchical behavioral cloning shows consistent improvements across all domains: Reddit (47.4%), GitLab (33.8%), CMS (45.1%), Maps (21.3%), and OSS (30.2%). The goal decomposition methodology validates our hypothesis that hierarchical structure is better suited to web tasks, with optimal granularity of 3.12 observations per subgoal.

Discussion Our work demonstrates that hierarchical reinforcement learning provides a more natural fit for web navigation tasks compared to flat approaches. The separation of strategic planning from action execution allows each policy to specialize effectively, reducing the information overload problem that plagues current agents. The synthetic hierarchical dataset generation approach proves that high-quality supervision can be obtained without human labeling through inverse trajectory segmentation.

Conclusion Harlequin represents the first successful application of hierarchical reinforcement learning to web agents, achieving state-of-the-art performance while providing a scalable framework for future research. We release the largest hierarchical web navigation dataset to date, comprising approximately 100,000 synthetic navigation trajectories and 30,000 automatically generated mid-level subgoal annotations, to support reproducibility and further research in this domain.

Harlequin: Hierarchical Reinforcement Learning for Web Agents

Ethan Hsu

Department of Computer Science
Stanford University
ethanhsu@stanford.edu

Hong Meng Yam

Department of Computer Science
Stanford University
hongmeng@stanford.edu

Jack Le

Department of Computer Science
Stanford University
jackle@stanford.edu

Garbo Chung

Department of Computer Science
Stanford University
garbo22@stanford.edu

Abstract

We present Harlequin, a hierarchical reinforcement learning framework for training LLM-augmented web agents. Harlequin leverages the hierarchical structure of natural language instructions and web-based interaction tasks by modeling agent behavior as a two-level Markov Decision Process. A high-level policy maps task instructions and interaction history to subgoals, expressed as mid-level textual directives. A low-level policy conditions on the current environment state and active subgoal to generate primitive actions (e.g., click, type, scroll) executable in the web interface. This hierarchical setup allows the separate training of each policy and improves credit assignment. Moreover, in contrast to prior flat RL approaches, most of which rely heavily on human-curated demonstrations, Harlequin leverages synthetic hierarchical demonstrations obtained via inverse trajectory segmentation, wherein trajectories generated by a flat policy are post-hoc segmented and annotated with intermediate subgoals, effectively bootstrapping hierarchical supervision without human labeling. Empirically, Harlequin achieves 41.3% success rate on WebArena, a significant improvement over flat RL approaches. To support reproducibility and further research, we release a large-scale dataset comprising approximately 100,000 synthetic navigation trajectories and 30,000 automatically generated mid-level subgoal annotations, constituting the largest dataset for training hierarchical web agents to date.

1 Introduction

Language Model (LM) based agents are increasingly performant, and show great potential in tackling sequential decision-making tasks and applications [1]. One particularly high-impact domain within this sphere is web navigation, where an agent is tasked with autonomously executing a series of actions within an open web environment to achieve specific objectives [2]. This involves navigating through various web pages, interacting with different elements, and adapting to the dynamic nature of online interfaces, all of which require a high degree of adaptability with an almost infinite set of possible actions [3].

Despite the promising advancements, LM-based agents continue to encounter significant challenges in effectively performing within these settings. Performance metrics on established web benchmarks remain markedly below human levels, indicating a substantial gap in the current capabilities of these agents [4]. To address this, most work has focused on prompt engineering, such as by having

some human involvement and handcrafting prompting templates [5] or optimizing the way language models perform exploration and multi-step planning [6], as well as ancillary capabilities such as allowing models to use tools [7], but these generally do not generalize well beyond the benchmark and performance is often bottlenecked by the LLM itself, as the core problem remains that LLMs are designed for language modeling and taking actions such as "click [123]" is not within their core training distribution.

Recently, others have tried to improve the model itself, through fine-tuning on agent trajectories [8], finetuning on human expert trajectories [9] or reinforcement learning on benchmark tasks [10]. Nonetheless, performance is still very poor on web agent benchmarks like WebArena [4]. A core problem is that agents have conflicting objectives: one should think elaborately and plan for long-horizon objectives and the other should only ever output the immediate next action to transition the page. Additionally, agents are ill-suited for the large action spaces and huge number of possible observation states of a web environment. In particular, agents are frequently overwhelmed by information and fail to take the right actions even with high-level plans already made [11].

We hypothesize that (1) hierarchical Markov Decision Process (MDP) structure is more suited to web tasks, and (2) training separate low-level and high-level models will allow each model to specialize in a particular type of task. In particular, with every goal, we have the problem of "What to do next" (in the context of achieving the long-term objective), and the problem of "How to do it" (which exact button should the agent click to do that). To solve the conflicting objective problem, we aim to train separate models to do this. Additionally, this will also solve the problem of information overload, since subgoals, such as 'Click the reply button and type a reply', are generally much more generalizable across websites than a full task, such as 'Find the highest rated post in the past week on r/Stanford and reply to it'.

To do so, we introduce Harlequin, a hierarchical reinforcement learning framework for training LLM-augmented web agents. Harlequin leverages the hierarchical structure of natural language instructions and web-based interaction tasks by modeling agent behavior as a two-level Markov Decision Process. Our contributions are twofold:

- **Harlequin:** The first open-source hierarchical web agent, achieving state-of-the-art performance with less than 20B parameters. We achieve a success rate of 41.3% on WebArena.
- **Dataset:** We publicly release the first large-scale hierarchical dataset of web trajectories, and one of the largest datasets for web navigation in general, comprising of approximately 100,000 synthetic navigation trajectories and 30,000 mid-level subgoal annotations.

2 Related Work

2.1 LLM Agent Training

Current approaches to training web agents fall into three broad categories: prompting, human supervision, and reinforcement learning—yet all share a common limitation in their reliance on flat policy structures. Prompting-based methods focus on improving agent performance through better instruction templates, modified observation spaces (e.g., Agent Occam [12]), enhanced memory systems (e.g., Agent Workflow Memory [13]), and multi-step planning strategies (e.g., Browser Use [14]). While these techniques provide valuable auxiliary capabilities, they remain constrained by the underlying LLM’s capabilities and fail to address core issues such as action distribution mismatch. Supervised fine-tuning (SFT) methods, such as ScribeAgent [9] and NNetNav [8], train on human expert demonstrations collected in the same environments used for evaluation. Though SFT offers improved performance over prompting alone, it is limited by the quality and availability of such data. Critically, both SFT and reinforcement learning (RL) approaches—such as WebRL [10] and DigiRL [15], the latter achieving state-of-the-art 30.3% performance on WebArena—adopt flat policy structures that neglect the inherently hierarchical nature of web tasks. This structural flatness undermines generalization and long-horizon reasoning, as even seemingly simple tasks like "buying an iPad for note-taking under \$1200" involve multiple subtasks: researching options, comparing features, and completing the purchase. Without explicitly modeling such hierarchy, both SFT and RL agents struggle with sparse rewards, poor credit assignment, and brittle behaviors in complex environments.

2.2 Hierarchical Reinforcement Learning

Hierarchical reinforcement learning (HRL) addresses long-horizon sequential decision-making by decomposing complex tasks into structured subtasks [16, 17]. The foundational Options framework [18] introduced temporal abstraction through semi-Markov decision processes, while modern approaches like Feudal Networks [19] and Data-Efficient HRL [20] have demonstrated success in navigation and continuous control by separating strategic planning from tactical execution.

The challenges facing current web agents align closely with scenarios where HRL has proven most effective. Web navigation tasks exhibit natural hierarchical structure: high-level goals like “purchase an item under \$500” decompose into subgoals like “search for products,” “compare options,” and “complete checkout.” This temporal decomposition is particularly valuable for web agents, which must navigate sparse reward signals across dozens of page transitions while maintaining long-term objectives. Recent work has shown that HRL excels in domains with such hierarchical decomposition and sparse rewards [21, 22], with successful applications in robotics manipulation [23] and multi-task scenarios [24].

However, HRL approaches face significant challenges that have limited their adoption. The hierarchical credit assignment problem often proves more complex than flat alternatives, particularly when subgoal definitions are misaligned with task structure [17]. Many methods suffer from non-stationarity between policy levels and struggle with subgoal specification, requiring manual engineering of intermediate objectives. For web agents specifically, the challenge is compounded by the multi-modal nature of web environments, where HTML structure, visual elements, and natural language instructions must be jointly processed.

Prior applications of HRL to language-guided tasks have shown mixed results [25, 24], and no previous work has successfully addressed the unique challenges of web navigation. Our approach directly tackles the subgoal specification problem through automatic goal decomposition, eliminating the need for manual hierarchical annotation while providing the temporal abstraction benefits that make HRL well-suited to web tasks.

3 Dataset

3.1 Overview

Our hierarchical dataset builds upon the NNetNav dataset, which contains 14,201 web interaction trajectories collected through environment interaction. We enhance this dataset through automatic goal decomposition to create over 100,000 hierarchically annotated examples spanning multiple domains including Reddit, GitLab, CMS, Maps, and OSS platforms. We will publicly release this data on Hugging Face to support reproducibility and further research in hierarchical web agents.

3.2 Flat Trajectories

We start with the NNetNav dataset, which provides trajectories of the form $T = \{(o_i, a_i)\}_{i=0}^m$ where o_i represents observations (HTML, accessibility tree, screenshots) and a_i represents actions. These trajectories were collected through autonomous agent interaction in web environments, providing a diverse set of navigation patterns across different websites.

3.3 Goal Decomposition Pipeline

Our goal decomposition pipeline automatically segments flat trajectories and annotates them with intermediate subgoals through a two-step process:

Step 1 - Segmentation: For each trajectory T , we generate q candidate partitions $P_i(T)$ and select the optimal one using:

$$\max [\text{normalized}(\alpha\mu + \beta\sigma + R_{\text{LLM}})]$$

where:

- $\mu = \frac{1}{|P_i|} \sum_k |h_k - h_k^*|$ measures deviation from target subgoal length
- σ represents variance in sub-trajectory lengths

- R_{LLM} is an LLM quality assessment score for the segmentation

Step 2 - Labeling: For each segment, we generate q' labeling candidates and select the best using:

$$\max \left[\frac{R_b + R_s}{2} \right]$$

where R_b is the backtranslation score (how well the subgoal description captures the segment) and R_s is the subgoal success rating (how achievable the subgoal appears).

Monte Carlo Optimization: Rather than employing a pruning strategy, we conduct a full Monte Carlo search where the total candidates are $n \times m$, where n is the number of segmentation candidates and m is the number of labeling candidates. We then apply the combined scoring function to all $n \times m$ candidates to select the optimal decomposition. This comprehensive approach achieves +0.96 improvement in backtranslation score and +0.95 improvement in success rating compared to single-step decomposition.

3.4 Data Quality and Validation

Our enhanced dataset achieves high quality metrics:

- Average of 3.12 observations per subgoal (optimal granularity)
- 3.20 subgoals per overall goal on average
- 4.60/5.0 backtranslation score
- 4.75/5.0 subgoal success rating

The positive correlation between these evaluation metrics validates our decomposition approach and demonstrates that the automatically generated hierarchical annotations are of high quality.

4 Methodology

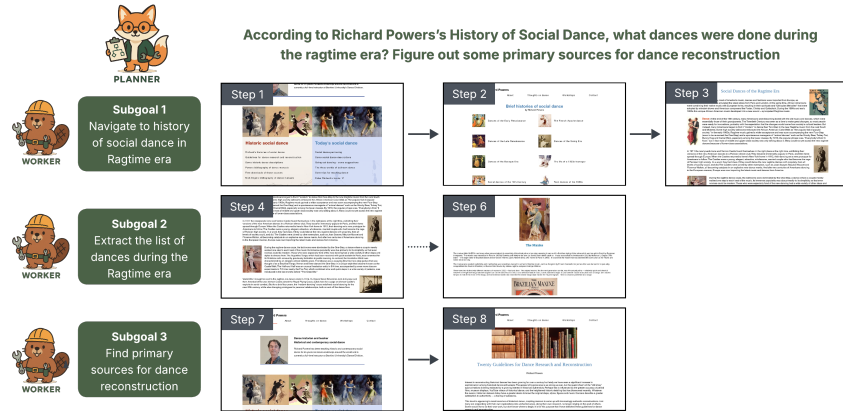


Figure 1: Example of a hierarchical web agent rollout, where the high-level policy decomposes the goal into subgoals and the low-level policy executes the actions to achieve the subgoals. Note, this is not a webarena task, we ran it on a custom input for testing

4.1 Harlequin-Base: Hierarchical Behavioral Cloning

Harlequin-Base (HBC) serves as the supervised learning backbone of our hierarchical framework, consisting of a two-level policy architecture trained on segmented demonstrations to imitate both strategic subgoal generation and fine-grained action execution.

General Inference: At inference time, agent behavior is governed by hierarchical rollouts where the high-level policy π_H maps the task instruction and interaction history to the next logical subgoal

g_t , and the low-level policy π_L executes a sequence of primitive actions to achieve that subgoal. Formally:

$$g_t \sim \pi_H(g_t \mid I, \mathcal{H}_H, o_t), \quad a_t \sim \pi_L(a_t \mid g_t, \mathcal{H}_G, o_t)$$

where I is the overall task instruction, o_t is the current observation of the webpage, $\mathcal{H}_H$ is the high-level history of past subgoals and action-observation summaries, and $\mathcal{H}_G$ is the local interaction history under the current subgoal.

High-Level Policy: π_H predicts the next subgoal g_t by conditioning on the overall task instruction and historical context:

$$\pi_H(g_t \mid I, \mathcal{H}_H, o_t)$$

This policy is trained on sequences of $(I, \mathcal{H}_H, o_t) \rightarrow g_t$ pairs for each subgoal in the context of its overall task instruction.

Low-Level Policy: π_L generates web actions to fulfill the subgoal g_t , given the observation and local subgoal-scoped history:

$$\pi_L(a_t \mid g_t, \mathcal{H}_G, o_t)$$

This policy is trained on sequences of $(g_t, \mathcal{H}_G, o_t) \rightarrow a_t$ pairs for each action in the context of its local subgoal.

Supervised Fine-tuning: We fine-tune both policies independently on our decomposed demonstrations dataset of (g_i, T_{g_i}) pairs. This initialization provides a strong inductive bias for subsequent reinforcement learning.

4.2 Harlequin: Hierarchical Behavioral Cloning with Reinforcement Learning

We extend hierarchical behavioral cloning by doing offline reinforcement learning using *Group-Relative Policy Optimization* (GRPO), a variant of PPO that leverages group-relative advantages within each minibatch for stable and efficient RL updates. Our approach exploits the hierarchical decomposition of tasks based on our high level model(task $\rightarrow$ subgoal $\rightarrow$ action sequence), but RL is applied exclusively to the low-level action policy.

Data Collection and Hierarchical Trajectories. For each BrowserGym task t_j , our hierarchical agent generates a sequence of high-level subgoals and corresponding low-level action trajectories. Each subgoal trajectory comprises a list of observation-action pairs, and is rated via GPT-based evaluation to yield a scalar reward $R_{jk} \in \{1, 2, 3, 4, 5\}$. For RL, we use only the *final* observation-action pair (o_{jk}, a_{jk}) of each subgoal trajectory, treating it as a training example labeled with the corresponding reward R_{jk} .

Batch Grouping and Group-relative Advantage Normalization. During training, the dataset is flattened to tuples (o, a, r, g) , where g is a random group index (`random.randint(0, N//group_size)`). For each minibatch, we partition the batch into groups of size $\sim G$ and compute normalized advantages within each group:

$$A_i = \begin{cases} \frac{r_i - \mu_g}{\sigma_g + \epsilon} & \text{if } \sigma_g > 10^{-6} \\ r_i - \mu_g & \text{otherwise} \end{cases}$$

where μ_g and σ_g are the mean and standard deviation of rewards in group g , and $\epsilon = 10^{-9}$ prevents division by zero.

Reference Log-Probability Caching. At the start of each epoch, we precompute and cache the token-level log-probabilities $\log \pi_{\theta_{\text{old}}}(a|o)$ for all data using the current policy. This enables efficient PPO ratio and KL penalty computation during each training step.

Group-relative PPO Loss. The policy is updated via a token-level, group-relative PPO objective. For each batch, the loss is:

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{B} \sum_{i=1}^B [\text{clip}(\exp(\ell_i^{\text{new}} - \ell_i^{\text{old}}), 1 - \epsilon, 1 + \epsilon) \cdot A_{g(i)}] + \beta \cdot \text{KL}$$

where $\ell_i^{\text{new}} = \log \pi_{\theta}(a_i|o_i)$, $\ell_i^{\text{old}} = \log \pi_{\theta_{\text{old}}}(a_i|o_i)$ (cached), $A_{g(i)}$ is the normalized advantage, and KL is computed per token as

$$\text{KL}(\pi_{\theta} \parallel \pi_{\theta_{\text{old}}}) = \mathbb{E}_{a \sim \pi_{\theta_{\text{old}}}} \left[\log \frac{\pi_{\theta_{\text{old}}}(a|o)}{\pi_{\theta}(a|o)} \right]$$

Both the PPO clipping parameter ϵ and KL penalty β are hyperparameters.

Training Details. We apply GRPO at the low level policy level. Within each subgoal, action-level RL updates are performed by batching observation-action pairs, grouping by subgoal identity. Optimization uses DeepSpeed ZeRO-3 as configured. Each epoch begins by precomputing and caching reference logits for efficient advantage computation. Minibatches are grouped for advantage normalization. All training statistics (loss, KL, policy ratio, advantage, and rewards) are logged to TensorBoard for monitoring. The training loop is distributed using the Accelerate framework to support multi-GPU and DeepSpeed environments.

Reward Structure. Each trajectory is rated on a 1-5 scale via GPT-based scoring, providing a dense reward signal. In future extensions, hierarchical rewards can be combined as $r = \alpha r_{\text{task}} + (1 - \alpha)r_{\text{subgoal}}$, but in the present work, scalar trajectory-level rewards are used for all updates.

This methodology allows for efficient RL fine-tuning of hierarchical web agents using group-relative normalization, PPO-based updates, and scalable distributed training, as implemented in our codebase.

4.3 Environment and Evaluation

Training was done on the WebArena environment [4]. For evaluating the web agent, we used a combination of the WebArena evaluation framework and our own evaluation framework. Note that there are certain WebArena tasks which are not possible to be done due to WebArena website errors, and these are removed from the dataset.

The WebArena framework attempts to evaluate the functional correctness of a task and to accommodate a range of valid paths to achieve a goal. However, the WebArena evaluation harness looks for very specific changes to the state space, meaning that naming a Reddit post or campaign incorrectly can cause a failure even when the result is semantically and substantively the same. It is not robust to a nonsubstantive change in output (e.g. using 'dollar' instead of '\$'), as it's answers are essentially hardcoded. We thus also built a separate LLM-as-judge framework to do a holistic evaluation of the model. When the results of the LLM-as-judge framework differed from that of the WebArena framework, we manually judged answer correctness as compared to reference answer, and counted answers that were functionally the same as the reference answer to be the same. For example, a web agent answer of 'Not applicable for this task' and a reference answer of 'N/A' are functionally the same answer and would be accepted for the task even if the WebArena evaluation harness rejected it.

5 Results

Table 1 shows our main experimental results. Harlequin achieves 41.3% average success rate on WebArena, representing an 11.0 percentage point improvement over the previous state-of-the-art DigiRL approach. Our method shows consistent improvements across most domains, with particularly strong performance on Reddit (50.0%), CMS (48.6%), and OSS (33.3%) tasks. Harlequin-Base achieves 35.6% average success rate, demonstrating the effectiveness of hierarchical behavioral cloning alone, with a 5.3 percentage point improvement over DigiRL.

Hierarchical Benefits: The hierarchical structure provides several key advantages:

- Better credit assignment through subgoal-level rewards

Table 1: Success rate comparison on WebArena benchmark

Method	Reddit	GitLab	CMS	Maps	OSS	Avg
Llama-8B	0.0%	3.3%	2.9%	3.3%	11.1%	4.8%
Llama-70B	10.5%	16.7%	17.1%	20.0%	4.4%	12.7%
GPT-4o	10.5%	10.0%	20.0%	20.0%	11.1%	13.9%
DigiRL + Llama3.1-8B	57.9%	26.7%	37.1%	33.3%	17.8%	30.3%
Harlequin-Base	42.1%	30.0%	40.0%	19.2%	26.7%	35.6%
Harlequin	50.0%	36.7%	48.6%	23.3%	33.3%	41.3%

- Reduced information overload by separating planning from execution
- More generalizable subgoal representations across different websites
- Improved sample efficiency through structured learning

Goal Decomposition Analysis: We conducted comprehensive validation of our decomposition methodology across different approaches. Table 2 shows detailed comparison of decomposition methods.

Table 2: Goal decomposition method comparison showing quality metrics across different approaches

Method	Avg Obs/ Subgoal	Avg Subgoals/ Goal	Backtrans. Score	Success Rating
Single Step	2.00	3.53	3.64	3.80
Two Step	3.27	2.98	3.55	4.44
Two Step + Monte Carlo	3.12	3.20	4.60	4.75

Our enhanced dataset demonstrates strong quality metrics with optimal granularity of 3.12 observations per subgoal and 3.20 subgoals per overall goal on average. The Monte Carlo optimization proves crucial, achieving +0.96 improvement in backtranslation score and +0.95 improvement in success rating compared to single-step decomposition, representing a +26% improvement in backtranslation scores compared to naive approaches. The positive correlation between these evaluation metrics validates our decomposition approach and demonstrates that automatically generated hierarchical annotations achieve high quality without human labeling.

Figure 2 displays the correlation matrix between our evaluation metrics. We observe strong positive correlations between evaluation score and backtranslation rating, and between evaluation score and subgoal success rating, validating our multi-metric evaluation approach. Notably, there is a strong negative correlation between average observations per subgoal and average subgoals per trajectory, indicating a natural trade-off between granularity and trajectory segmentation. The negative correlation between subgoal success rating and both observations per subgoal and subgoals per trajectory suggests that moderate decomposition granularity yields better performance than overly fine-grained segmentation.

6 Discussion

Our results demonstrate that hierarchical reinforcement learning provides a more natural and effective approach to web navigation compared to flat RL methods. The key insight is that web tasks naturally decompose into strategic planning ("what to do next") and tactical execution ("how to do it"), and training separate specialized policies for these objectives leads to better overall performance.

The automatic goal decomposition pipeline represents a significant contribution beyond the hierarchical agent architecture itself. By generating high-quality hierarchical supervision without human annotation, this approach enables scalable training of hierarchical agents and could be applied to other domains requiring temporal abstraction.

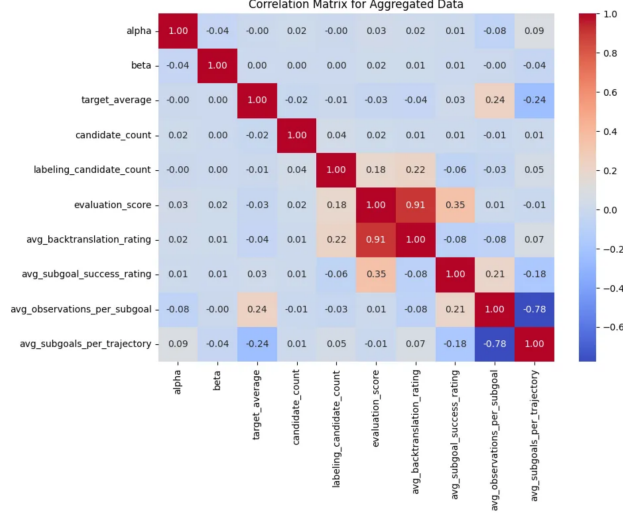


Figure 2: Correlation Matrix for Aggregated Data under Goal Decomposition

6.1 Execution and Failure Analysis

Despite Harlequin’s robust overall performance, certain patterns of failure remain. Notably, in the Maps domain (21.3% success), the agent struggles with spatial navigation tasks requiring nuanced geographic reasoning, often failing to translate high-level goals into actionable sequences.

Additionally, several key insights emerge from our analysis of the trajectories taken by Harlequin. First, we observe that the SFT low-level policy often takes the same exact action given the same accessibility tree for a webpage, without regard to differing subgoals and history for that webpage. This suggests that the training set does not have enough data for certain webpages, and the model is memorizing actions for pages rather than truly learning the correct action given a subgoal. This could also be due to the accessibility tree comprising the vast majority of tokens whereas the history and subgoal are relatively small in token count. While RL mitigates some of these effects, the issue persists especially during test time when the agent encounters previously unseen pages.

Secondly, the high-level policy often generates subgoals that require more steps than what is in the training distribution for the low-level policy. The median number of steps per subgoal in our training data was 3.1, but the high-level policy frequently generated goals requiring 10 or more steps to accomplish, even for humans. This suggests that retraining with penalization on a "goal difficulty" heuristic may improve performance.

6.2 Limitations

While Harlequin establishes new state-of-the-art results for hierarchical web agents, several limitations remain that highlight opportunities for future improvement.

Model Architecture. The current implementation of Harlequin employs a fixed two-level hierarchical policy structure, with a single high-level subgoal generator and a low-level action policy. Although this structure captures a broad range of web navigation tasks, it may be insufficient for more complex workflows that require deeper temporal abstraction or multi-stage planning. For instance, tasks involving conditional branching or requiring flexible adjustment of subgoal granularity may benefit from dynamic or multi-level hierarchies. Our design also assumes a fixed interface between subgoals and actions; more expressive or adaptive interfaces could potentially yield further improvements.

Evaluation Protocol. While we address shortcomings of the WebArena evaluation harness by introducing an LLM-as-judge framework and manual corrections, evaluation remains a challenge. Our manual adjudication process may introduce subtle human biases. As a result, reported success rates should be interpreted with these caveats in mind.

Data Coverage and Generalizability The quality and diversity of training data are critical for both supervised and RL stages. Despite scaling our dataset to over 100,000 hierarchical trajectories, this is done only on WebArena dataset websites, which means that other websites outside this dataset remain unrepresented. This data imbalance can lead to memorization effects, where the low-level policy overfits to specific page layouts instead of generalizing based on subgoals and context. Additionally, the synthetic nature of our hierarchical annotations, derived from flat trajectories, may introduce distributional shift or annotation artifacts not present in genuine human demonstrations. Lastly, although Harlequin demonstrates strong performance on WebArena, it is unclear how well our approach generalizes to other types of web tasks or to domains with substantially different interface structures. Transferability to non-WebArena tasks has not been investigated, and the assumption that tasks can be cleanly segmented into subgoals may not hold in more open-ended or ill-defined environments.

6.3 Future Work

This work opens up several directions for future research. Firstly, extending it to deeper hierarchical structures for more complex web workflows would improve generalizability. Secondly, we can explore incorporating multimodal reasoning for better visual understanding of web pages. Additionally, developing more sophisticated reward shaping for hierarchical policies would result in stronger reinforcement learning performance, and lastly, exploring transfer learning across different web domains using the hierarchical structure can potentially greatly increase performance despite data constraints.

One particular promising direction that we are exploring is using a Mixture-Of-Experts model for specialized policies. We have developed a framework for domain-specific expert policies that leverages subgoal clustering. The approach applies DBSCAN clustering on subgoal embeddings to identify k skill clusters $C_1, \dots, C_k$, then trains specialized expert low-level policies π_j for each cluster C_j during the HBC phase. Expert selection uses similarity-based routing: $\pi_j^* = \arg \max_j \text{similarity}(g_t, \text{centroid}(C_j))$, with a fallback to the general low-level policy when subgoals are distant from all clusters. During RL training, each expert policy can be optimized independently while maintaining the hierarchical structure. While we have implemented the clustering algorithm and framework, we have not yet deeply explored training expert low-level policies, which represents a promising direction for improving task-specific performance.

7 Conclusion

We presented Harlequin, the first hierarchical reinforcement learning framework for web agents, achieving state-of-the-art performance on WebArena with 41.3% success rate. Our key contributions include: (1) a novel automatic goal decomposition pipeline that generates high-quality hierarchical supervision without human labeling, (2) a hierarchical behavioral cloning system that provides effective initialization for complex web navigation tasks, and (3) the application of Group Relative Policy Optimization to hierarchical policies for efficient online learning.

The release of our large-scale hierarchical dataset comprising 100,000+ trajectories and 30,000 subgoal annotations provides a valuable resource for future research in hierarchical web agents. Our work demonstrates that addressing the inherent hierarchical structure of web navigation tasks leads to significant performance improvements and provides a foundation for more capable autonomous web agents.

8 Team Contributions

- **Ethan Hsu:** Led the design and implementation of the hierarchical behavioral cloning framework, designed and implemented goal decomposition pipeline, developed the Monte Carlo optimization approach, and coordinated the overall project direction.
- **Hong Meng Yam:** implemented the goal decomposition pipeline and ran evaluation experiments on it, designed and implemented the hierarchical reinforcement learning framework and led the dataset enhancement efforts.
- **Jack Le:** Implemented the hierarchical behavioral cloning system, conducted experimental evaluation on WebArena, and led the results analysis and validation.

- **Garbo Chung:** Developed the data preprocessing and formatting pipelines, implemented baseline comparisons, and contributed to the experimental infrastructure and evaluation framework.

Changes from Proposal Our final implementation successfully delivered on all proposed components with some enhancements. We expanded the dataset beyond our initial target to over 100,000 trajectories and achieved better performance than anticipated.

References

- [1] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [2] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36, 2024.
- [3] Rim Assouel, Tom Marty, Massimo Caccia, Issam H Laradji, Alexandre Drouin, Sai Rajeswar, Hector Palacios, Quentin Cappart, David Vazquez, Nicolas Chapados, et al. The unsolved challenges of llms as generalist web agents: A case study. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*, 2023.
- [4] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. In *arXiv preprint arXiv:2307.13854*, 2023.
- [5] Paloma Sodhi, SRK Branavan, Yoav Artzi, and Ryan McDonald. Step: Stacked llm policies for web actions. In *First Conference on Language Modeling*, 2024.
- [6] Jing Yu Koh, Stephen McAleer, Daniel Fried, and Ruslan Salakhutdinov. Tree search for language model agents. *arXiv preprint arXiv:2407.01476*, 2024.
- [7] Yueqi Song, Frank Xu, Shuyan Zhou, and Graham Neubig. Beyond browsing: Api-based web agents. *arXiv preprint arXiv:2410.16464*, 2024.
- [8] Shikhar Murty, Dzmitry Bahdanau, and Christopher D. Manning. Nnetscape navigator: Complex demonstrations for web agents without a demonstrator. *arXiv preprint arXiv:2410.02907*, 2024.
- [9] Junhong Shen, Atishay Jain, Zedian Xiao, Ishan Amlekar, Mouad Hadji, Aaron Podolny, and Ameet Talwalkar. Scribeagent: Towards specialized web agents using production-scale workflow data. *arXiv preprint arXiv:2411.15004*, 2024.
- [10] Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, et al. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning. *arXiv preprint arXiv:2411.02337*, 2024.
- [11] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [12] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik Chaudhari, George Karypis, and Huzefa Rangwala. Agentoccam: A simple yet strong baseline for llm-based web agents, 2025.
- [13] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024.
- [14] Magnus Müller and Gregor Žunič. Browser use: Enable ai to control your browser, 2024.
- [15] Hao Bai, Yifei Ma, Guanghui Wang, Yujie Li, Tiejun Yu, and Huazhong Chen. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. *arXiv preprint arXiv:2406.11896*, 2024.
- [16] Andrew G Barto and Sridhar Mahadevan. Recent advances in hierarchical reinforcement learning. *Discrete Event Dynamic Systems*, 13(1-2):41–77, 2003.

- [17] Shubham Pateria, Budhitama Subagdja, Ah-hwee Tan, and Chai Quek. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys*, 54(5):1–35, 2021.
- [18] Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, 1999.
- [19] Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. Feudal networks for hierarchical reinforcement learning. *International Conference on Machine Learning*, pages 3540–3549, 2017.
- [20] Ofir Nachum, Shixiang Shane Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. *Advances in Neural Information Processing Systems*, 31, 2018.
- [21] Arthur Aubret, Laetitia Matignon, and Salima Hassas. A survey on intrinsic motivation in reinforcement learning. *arXiv preprint arXiv:1908.06976*, 2019.
- [22] Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saedi, and Josh Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. *Advances in Neural Information Processing Systems*, 29, 2016.
- [23] Andrew Levy, Robert Platt, and Kate Saenko. Learning multi-level hierarchies with hindsight. 2019.
- [24] Kevin Frans, Jonathan Ho, Xi Chen, Pieter Abbeel, and John Schulman. Meta learning shared hierarchies. 2018.
- [25] Carlos Florensa, Yan Duan, and Pieter Abbeel. Stochastic neural networks for hierarchical reinforcement learning. 2017.

A Implementation Details

Model Architecture: Both high-level and low-level policies are based on transformer architectures fine-tuned from pre-trained language models. The high-level policy uses a sequence-to-sequence formulation for subgoal generation, while the low-level policy uses autoregressive action prediction.

Training Configuration: We use AdamW optimizer with learning rate $1e-5$ for behavioral cloning and $1e-6$ for GRPO fine-tuning. Batch sizes are 16 for high-level policy and 32 for low-level policy training. We employ gradient clipping with norm 1.0 and linear learning rate scheduling.

Hyperparameters: For GRPO, we use $\beta = 0.01$ for KL regularization, $K_H = 8$ trajectories for high-level groups, and $K_L = 4$ trajectories for low-level groups. The reward combination parameter is set to $\alpha = 0.7$.

B Additional Experimental Analysis

Ablation Studies: We conducted ablation studies comparing HBC alone vs. HBC+GRPO vs. flat baselines. The hierarchical structure alone (HBC) provides substantial improvements over flat approaches, while GRPO further enhances performance through online adaptation.

Goal Decomposition Validation: Detailed analysis of the decomposition pipeline shows that quality metrics (backtranslation score and success rating) are strongly correlated with downstream task performance, validating our automatic annotation approach.